

# AI Form Responder Automation

## Employer-Facing Project Case Study

Microsoft Forms, Azure Logic Apps, Azure OpenAI, Azure AI Foundry, and Outlook automation

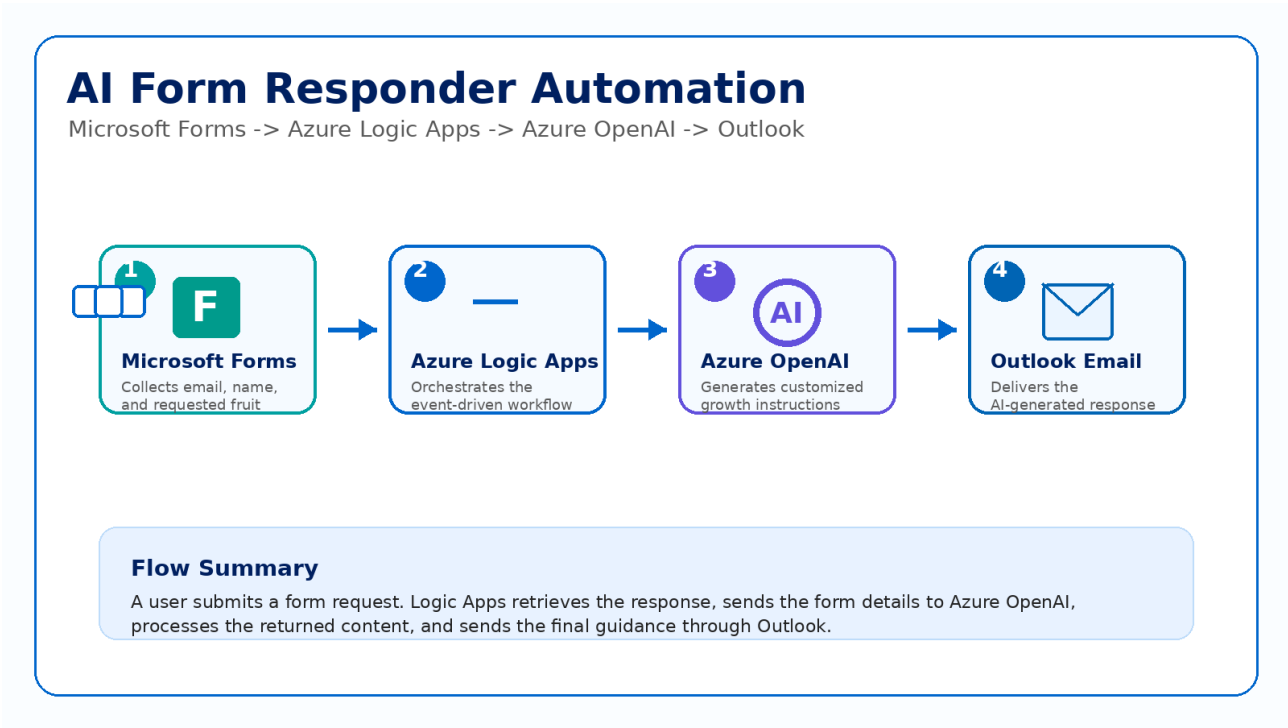


Figure 1. High-level architecture of the AI Form Responder Automation.

|                    |                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------|
| Project Type       | AI-powered cloud automation / Microsoft 365 workflow project                                          |
| Business Value     | Automatically generates customized form responses and sends them by email                             |
| Core Services      | Microsoft Forms, Azure Logic Apps, Azure OpenAI / Foundry, Outlook, API Connections                   |
| Documentation Note | Sensitive identifiers, URLs, emails, and subscription details should remain blurred before publishing |

Prepared as a portfolio-ready case study. Replace live links with this document or a static project page when publishing online.

## 1. Executive Summary

The AI Form Responder Automation is a cloud workflow that receives a Microsoft Forms submission, uses Azure OpenAI to generate customized guidance, and sends the completed response to the user through Outlook.

The sample business scenario is a Fruit Growth Expert form. A user enters their email, full name, and the fruit they want to grow. The workflow reads those answers, asks the AI model to generate useful growing instructions, and emails the response without manual work.

The revised report keeps the explanation concise while still showing the business value, architecture, Azure service responsibilities, workflow execution, and employer-facing skills demonstrated by the project.

## 2. Business Problem and Solution

Manual form follow-up is repetitive and slow. A business user may collect requests through Microsoft Forms, but responding to every submission individually creates delay and inconsistent output.

This solution automates that process. The form captures the request, Logic Apps orchestrates the workflow, Azure OpenAI generates the custom response, and Outlook sends the final message. The result is a repeatable AI-assisted response process that can be adapted for customer support, training requests, internal helpdesk intake, or onboarding workflows.

## 3. Service Responsibility Matrix

| Service                | Responsibility                                                            | Portfolio Value                                           |
|------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------|
| Microsoft Forms        | Collects the email, full name, and requested fruit.                       | Shows user input capture and Microsoft 365 integration.   |
| Azure Logic Apps       | Orchestrates the workflow from form submission to email delivery.         | Shows low-code cloud workflow automation.                 |
| Azure OpenAI / Foundry | Generates personalized fruit-growing instructions using a deployed model. | Shows generative AI integration with business automation. |
| Outlook Connector      | Sends the AI-generated instructions to the user.                          | Shows communication automation using managed connectors.  |
| API Connections        | Authenticate and connect Forms, Outlook, and Azure OpenAI actions.        | Shows connector configuration and service integration.    |

4. Architecture and End-to-End Flow

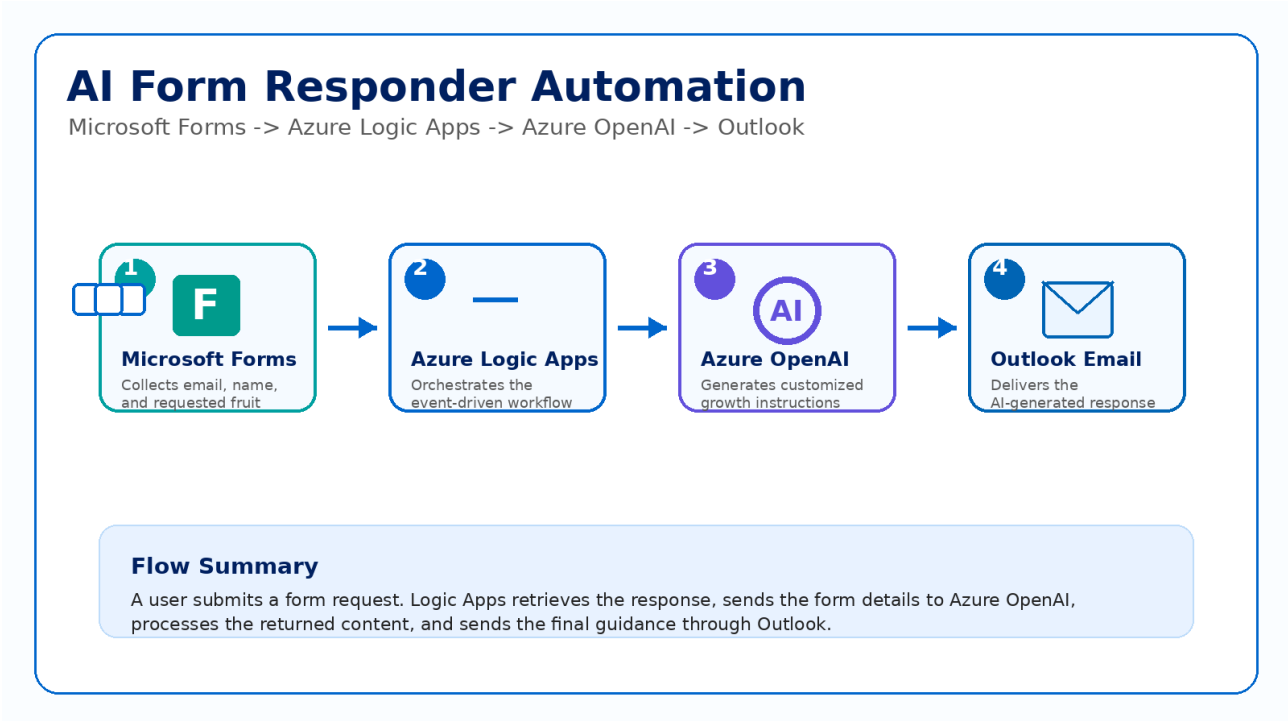


Figure 2. Microsoft Forms triggers Logic Apps, Azure OpenAI generates content, and Outlook sends the email.

The architecture is intentionally simple and employer-friendly: Microsoft Forms captures the request, Azure Logic Apps controls the workflow, Azure OpenAI generates the response, and Outlook delivers the final email. This proves the project is more than a single-tool demo; it is an integrated automation across Microsoft 365 and Azure AI services.

|                       |                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>        | Show the project architecture at a glance.                                                               |
| <b>What it proves</b> | Demonstrates an integrated workflow across Microsoft Forms, Azure Logic Apps, Azure OpenAI, and Outlook. |

5. Azure Resource Design and Cost Control

The original resource group screenshot documents the deployed Logic App, API connections for Microsoft Forms and Outlook, the Azure OpenAI / Foundry resource, and the Foundry project. Grouping these services in one resource group makes the solution easier to manage, monitor, and clean up.

The Logic App overview also confirms the workflow configuration: one trigger, four actions, stateful execution tracking, and disabled status after testing. Disabling the workflow after documentation prevents unnecessary runs and demonstrates responsible cloud cost management.

|                       |                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>        | Document all cloud resources used by the solution and show the workflow state.                                 |
| <b>What it proves</b> | Demonstrates Azure resource organization, awareness of dependencies, run tracking, and post-demo cost control. |

6. Logic App Workflow Execution

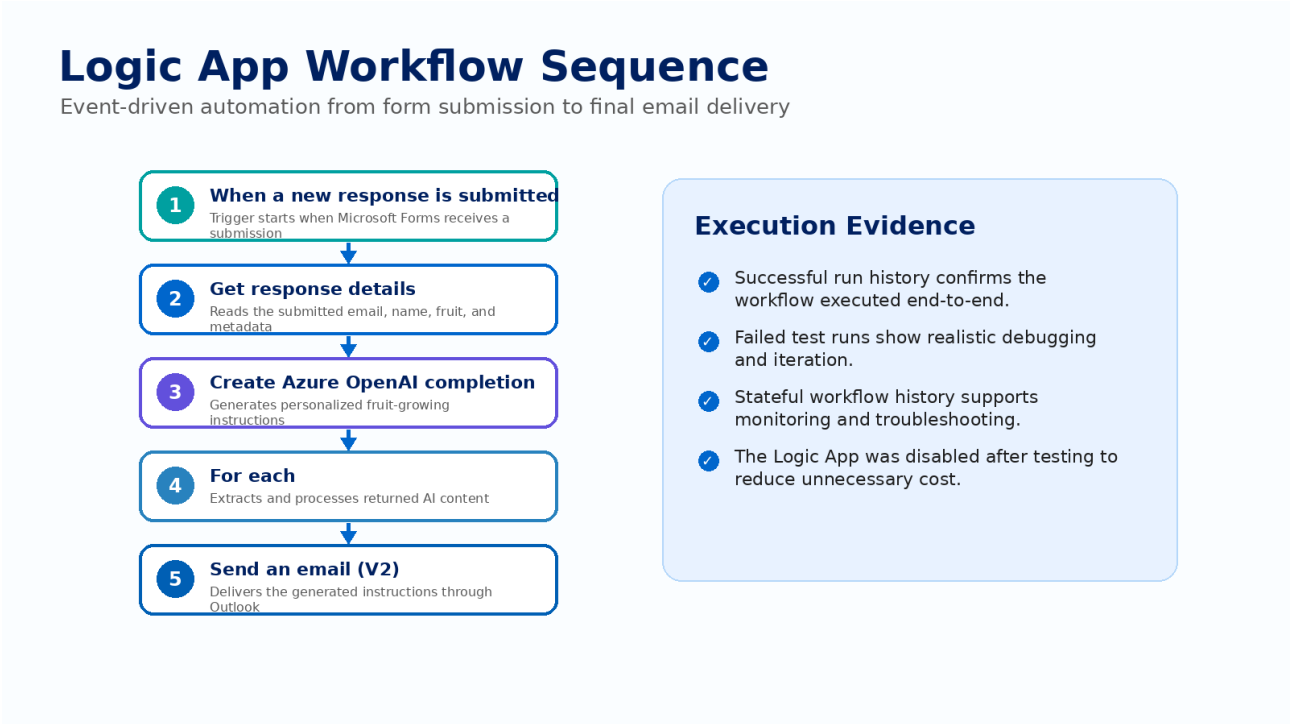


Figure 3. Logic App workflow sequence from form trigger to final email delivery.

The workflow begins when a new Microsoft Forms response is submitted. Logic Apps then retrieves the full response details, sends the submitted values into the Azure OpenAI completion action, processes the returned AI output, and sends the generated instructions through the Outlook connector.

The run history confirms that the automation executed successfully end-to-end. Failed test runs are also useful evidence because they show realistic troubleshooting and iteration during development.

|                       |                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>        | Show the full successful execution path and monitoring evidence.                                       |
| <b>What it proves</b> | Demonstrates workflow diagnostics, run history, iterative testing, and validation of cloud automation. |

7. Microsoft Forms Input and Dynamic Data Extraction

The Microsoft Form titled Fruit Growth Expert is the user-facing entry point. It collects the user email, full name, and the fruit they want to grow. In the documented sample, the requested fruit was Apples.

After the trigger fires, the Get response details action retrieves the submitted values. These values become dynamic content for both the AI prompt and the final email response. This is the key step that connects the user input to the AI-generated output.

|                       |                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>        | Show how user input starts the automation and becomes workflow data.                                      |
| <b>What it proves</b> | Demonstrates Microsoft Forms integration, dynamic content extraction, and event-driven automation design. |

## 8. Azure OpenAI Completion and Response Processing

The Azure OpenAI completion action generates personalized fruit-growing instructions from the submitted form data. In the sample workflow, the model returns guidance for growing apples, including practical steps such as choosing a variety, selecting a planting location, and preparing the soil.

The For each block processes the content returned from the AI action before email delivery. This is useful when the response payload contains structured or array-based output that must be extracted before being passed into the Outlook email action.

|                       |                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>        | Document how the workflow generates and prepares the AI response.                                                          |
| <b>What it proves</b> | Demonstrates AI service integration, prompt-driven content generation, and structured response handling inside Logic Apps. |

## 9. Final Email Output

The final Outlook email proves that the workflow produced a complete user-facing result. The documented email used the subject “Here are your growth instructions” and included an AI-generated body titled “How to Grow Apples.”

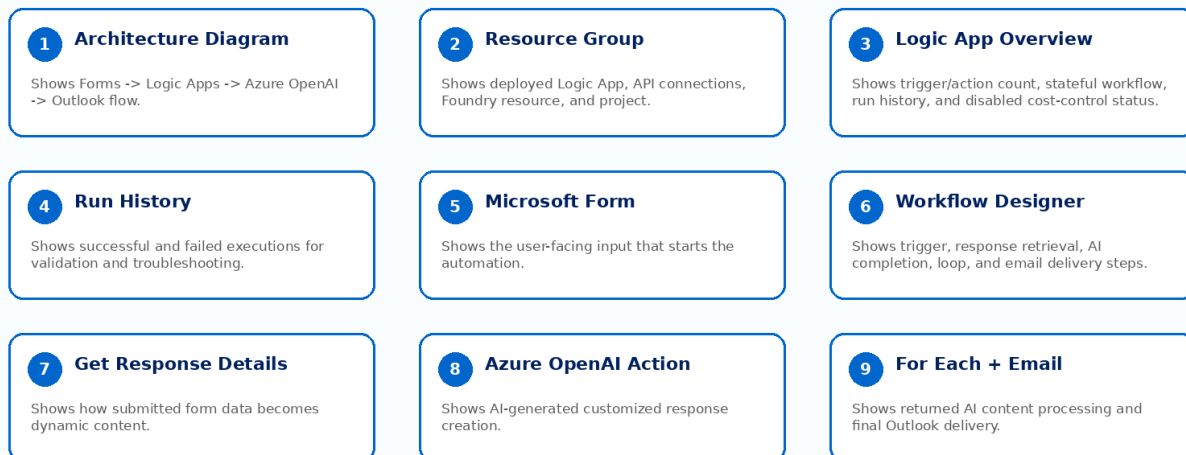
This final output is important because it closes the business loop: the user submits a form, the system processes the request, AI generates the answer, and the completed response is delivered back to the user automatically.

|                       |                                                                                                     |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| <b>Purpose</b>        | Show the final user-facing result of the automation.                                                |
| <b>What it proves</b> | Demonstrates successful output delivery, email automation, and completion of the business workflow. |

## 10. Screenshot Documentation Map

### Screenshot Evidence Map

Concise description of each portfolio screenshot and what it proves



*Figure 4. Concise map of the screenshots and the evidence each one provides.*

This evidence map summarizes the documentation value of the original screenshots. Each screenshot should remain in the final portfolio package only after sensitive details such as email addresses, subscription IDs, workflow URLs, API connection details, and account names are blurred or cropped.

## 11. Skills Demonstrated to Employers

**Cloud workflow automation:** Built an event-driven workflow using Azure Logic Apps and Microsoft 365 connectors.

**Generative AI integration:** Connected form data to an Azure OpenAI completion action to produce customized content.

**Microsoft 365 integration:** Used Microsoft Forms for input collection and Outlook for automated response delivery.

**Debugging and monitoring:** Used run history and execution details to validate successful and failed runs.

**Responsible cloud management:** Disabled the Logic App after testing and documented cleanup recommendations.

**Portfolio communication:** Documented the business use case, architecture, workflow, and proof of execution clearly.

## 12. Security, Privacy, and Publishing Notes

- Blur or crop live workflow URLs, callback URLs, API connection details, subscription IDs, tenant/account information, and real email addresses.
- Do not publish the live Logic App endpoint or any screenshots that expose connection strings, tokens, keys, or user-identifying information.
- For the portfolio, replace live demo links with a “View Case Study” or “View Documentation” button.
- Keep the Logic App disabled when not actively testing to avoid unnecessary executions and potential cost.

## 13. Future Improvements

- Add input validation and error handling for missing or invalid form responses.
- Store each request and response in a database for auditing and reporting.
- Add Application Insights or Log Analytics for richer monitoring and alerting.
- Parameterize the AI prompt so the same workflow can support multiple form types.
- Create a cleaner public-facing frontend that calls the workflow without exposing live endpoints.

## 14. Conclusion

The AI Form Responder Automation is a strong portfolio project because it connects a real business problem with a practical Microsoft cloud solution. It shows how form submissions can be turned into AI-generated responses and delivered automatically by email.

The revised case study presents the project in a cleaner Azure-themed format that matches the Email Reminder Agent documentation: concise sections, clear figure descriptions, service responsibility mapping, privacy notes, and employer-focused value statements.

This project is especially relevant for Power Platform, Azure integration, cloud automation, and AI-enabled business workflow roles.